

This code is a classic implementation of an **Asynchronous FIFO** that uses a "dual-gray-code" approach to handle two independent clock domains safely.

Here is the absolute deep-dive into every section of the code, explaining the specific hardware logic behind the lines.

1. Library and Entity Breakdown

- **Libraries:** Standard IEEE libraries are used for logic and arithmetic.
 - **Generics:**
 - ADDR_BITS (default 9): Sets the addressable depth to $2^9 = 512$.
 - DATA_BITS (default 32): Sets the width of the data bus.
 - **Write Port:** Operates on wclk_i. Includes wreq_i (write request), wdata_i (input data), wfull_o (indicates no more room), and wdcount_o (shows how many words are in the FIFO from the write perspective).
 - **Read Port:** Operates on rclk_i. Includes rreq_i (read request), rdata_o (output data), rempty_o (indicates no data available), and rdcount_o (shows how many words are in the FIFO from the read perspective).
-

2. The XOR Reduction Function

```
function XOR_Reduction(data: in std_logic_vector) return std_logic is ...
```

This is a recursive XOR gate in software form.

- **Logic:** It takes a vector and XORs all bits together: $\text{bit}_0 \oplus \text{bit}_1 \oplus \text{bit}_2 \dots$.
 - **Purpose:** This is the mathematical key to converting Gray code back to Binary. To find bit i of a Binary number from a Gray number, you XOR all bits of the Gray number from i to the MSB.
-

3. Pointer Logic & The "Extra Bit"

The pointers (wbin_r, rbin_r, etc.) are declared as ADDR_BITS downto 0 (10 bits for a 9-bit address).

- **The 10th Bit:** This is the "wrap-around" bit. If wbin_r is 011111111 (511) and increments, it becomes 1000000000 (512).
- **Why?:** If the pointers are identical (e.g., both 0), the FIFO is **Empty**. If the pointers are

identical except for the MSB (e.g., Write is 512, Read is 0), the FIFO is **Full**.

4. Sequential Processes (The Hardware Registers)

Write Clock Domain (REG_WRT_WCLK_I)

- **Reset:** Asynchronously clears write pointers and status.
- **Synchronization:** $wrptr_meta \leq rptr_r$; and $wrptr_sync \leq wrptr_meta$;,. This is a **2-stage synchronizer**. It brings the Gray-coded read pointer into the write clock domain while protecting against metastability.
- **Updating:** Every clock cycle, it moves the "next" pointer values to the current registers.

Read Clock Domain (REG_WRT_RCLK_I)

- **Reset:** Sets $rempty_o$ to '1' (because a reset FIFO is empty).
- **Synchronization:** Brings the Gray-coded write pointer into the read domain via $rwptr_meta$ and $rwptr_sync$.

Memory Access (RAM_WR & RAM_RD)

- **Write:** Performs $ram(address) \leq wdata_i$ when $wreq_i$ is active. It truncates the 10-bit pointer to 9 bits for the actual RAM address.
 - **Read:** Reads data into $rdata_o$ when $rreq_i$ is active.
-

5. Combinational Logic (The Math)

Gray Code Conversion

- $wptr_nxt \leq (wbin_nxt \gg 1) \text{ XOR } wbin_nxt$;
- **The Benefit:** Only one bit changes at a time in Gray code. This ensures that when the synchronizer in the other clock domain samples the pointer, it never sees a transition where multiple bits are flipping, which would cause a massive error.

Full/Empty Condition

- **Empty:** If the Gray write pointer matches the Gray read pointer exactly, $rempty_nxt$ is high.
- **Full:** $wfull_nxt$ is high if the write pointer's top two bits are inverted relative to the synchronized read pointer, but the rest match. This signifies the write pointer has wrapped around exactly once and "caught" the read pointer.

Data Occupancy (WDCOUNT_CAL & RDCOUNT_CAL)

This section calculates how many words are currently stored.

1. **Gray to Binary:** Uses the XOR_Reduction loop to turn the synchronized pointers back into binary ($wbin$ and $rwbin$).

2. Difference Calculation:

- If the write pointer is ahead of the read pointer, it's a simple subtraction.
- If the write pointer has wrapped around (different "lap"), it calculates the distance by adding the DEPTH to the difference.

Logic Summary Table

Feature	Implementation	Purpose
Storage	array of std_logic_vector	Holds the 32-bit data words.
Safety	Gray Coding	Prevents multi-bit transition errors across clocks.
Stability	2-Stage Flip-Flops	Prevents metastability during clock domain crossing.
Tracking	wdcount_o / rdcount_o	Real-time monitoring of how full the FIFO is.

This asynchronous FIFO (First-In, First-Out) is a sophisticated piece of digital logic designed to pass data between two different clock domains safely. Below is the detailed breakdown of the internal signals, the data flow, and the operational logic.

Internal Signals & State Tracking

The architecture uses several specialized signals to manage the transition between the **Write Domain** (`wclk_i`) and the **Read Domain** (`rclk_i`).

- **Binary Pointers** (`wbin_r`, `rbins_r`): These are $\$ADDR_BITS + 1$ (10 bits for a 9-bit address). The extra most significant bit (MSB) acts as a "wrap-around" indicator to distinguish whether the FIFO is completely full or completely empty.
- **Gray Code Pointers** (`wptr_r`, `rptra_r`): These are the binary pointers converted to Gray code. Because only one bit changes at a time in Gray code, they can be safely sampled by the opposing clock domain without risk of seeing multi-bit transition errors.
- **Synchronizer Chain** (`wrptr_meta/sync`, `rwptr_meta/sync`): These are two-stage flip-flop chains. They "buffer" the Gray code pointers as they cross from one clock domain to the other, reducing the probability of metastability failures.
- **wrbins** and **rwbin**: These signals hold the binary version of the *other* domain's pointer

after it has been synchronized and converted back from Gray code. These are used specifically to calculate the data count (occupancy).

The Data Flow Path

1. **Data Input:** Data enters via `wdata_i` and is written into the `ram` array at the address specified by the lower bits of `wbin_r`.
 2. **Write Pointer Update:** When a write request (`wreq_i`) is made, `wbin_nxt` increments the binary pointer. This is then converted to the Gray code `wptr_nxt`.
 3. **Cross-Domain Synchronization:** The Gray-coded write pointer (`wptr_r`) is passed to the read domain, where it is stabilized through two registers (`rwptr_meta` and `rwptr_sync`).
 4. **Status Comparison:**
 - **In the Read Domain:** The stabilized write pointer is compared against the local read pointer to determine if the FIFO is empty (`empty_o`).
 - **In the Write Domain:** The stabilized read pointer is compared against the local write pointer to determine if the FIFO is full (`wfull_o`).
 5. **Data Output:** When a read request (`rreq_i`) is made, the data is pulled from the `ram` at the address specified by `rbn_r` and sent to `rdata_o`.
-

Core Working Logic

1. Handling the "Full" vs. "Empty" Ambiguity

A common problem in circular buffers is that the pointers are equal when the buffer is 0% full and 100% full. This code uses an extra MSB to solve this:

- **Empty:** The FIFO is empty when the Gray write pointer exactly matches the Gray read pointer.
- **Full:** The FIFO is full when the write pointer has wrapped around (different MSB) but the remaining address bits match the read pointer.

2. Gray-to-Binary Conversion for Occupancy

While Gray code is great for safety, you cannot easily do math (subtraction) with it to find out how many items are in the FIFO. The code implements a `XOR_Reduction` function to convert the synchronized Gray pointers back into binary (`wbin` and `rbn`).

3. Data Count Calculation

The counts (`wcount_o` and `rdcount_o`) are calculated by subtracting the two binary pointers:

- **Simple Case:** If both pointers are in the same "lap" (the MSBs match), it is a simple subtraction: `$WritePointer - ReadPointer$`.

- **Wrap-Around Case:** If the pointers are in different "laps" (MSBs differ), it accounts for the rollover by using the **DEPTH** constant: $\$DEPTH - (\text{ReadPointer} - \text{WritePointer})\$$.

Logic Summary Table

Process	Domain	Function	Logic Type
Pointer Increment	Both	<code>wbin_nxt / rbin_nxt</code>	Sequential
RAM Storage	Write	<code>ram(addr) <= wdata_i</code>	Sequential
Clock Crossing	Both	2-Stage Sync (<code>meta</code> -> <code>sync</code>)	Sequential
Full Generation	Write	MSB Inversion check	Combinational
Empty Generation	Read	Pointer match check	Combinational
Occupancy	Both	<code>XOR_Reduction</code> + Subtraction	Combinational

1. Block-by-Block Logic

This design is split into two "islands" of logic: the **Write Domain** and the **Read Domain**, connected by a "bridge" of synchronizers.

The Memory Block (RAM)

The core storage is a dual-port RAM array.

- **Write Side:** When `wreq_i` is high, data is written into the RAM using the binary write pointer as the address.
- **Read Side:** When `rreq_i` is high, data is pulled out using the binary read pointer.

The Pointer & Gray Code Block

This block manages where data goes in and out.

- **Binary Increment:** Every time you write or read, a binary counter (`wbin_r` or `rbin_r`) increments.
- **Binary to Gray Conversion:** The binary value is converted to **Gray Code** (`wptr_nxt / rptr_nxt`).
- **Why?** In binary, changing from 7 (`0111`) to 8 (`1000`) flips four bits at once. If the other clock domain samples the signal at that exact micro-moment, it might see a completely wrong number. In Gray code, only **one bit** ever changes at a time, making it safe to

"cross" the clock boundary.

The Synchronizer Block

Since the clocks are independent, we can't just send a signal from one side to the other. We use a **two-stage flip-flop chain** (*meta* and *sync* signals). This "settles" the signal in the new clock domain, preventing a hardware state called **metastability** (where a bit gets stuck between '0' and '1').

2. Teaching XOR Reduction

The *XOR_Reduction* function in the code is a mathematical tool used to convert Gray code back into a standard Binary number.

How it works:

In digital logic, to find bit i of a Binary number from a Gray code, you must XOR the i -th bit of the Gray code with all the bits to its left (more significant bits).

The Code's Implementation:

VHDL

```
for i in 0 to ADDR_BITS loop
  TEMP := '0';
  for j in i to ADDR_BITS loop -- Conceptually what happens inside XOR_Reduction
    TEMP := TEMP XOR data(j);
  end loop;
end loop;
```

By performing this "reduction" (XORing a range of bits down to a single bit), the code reconstructs the binary pointers *wrbn* and *rwbn*. Without this conversion, the FIFO wouldn't be able to calculate exactly how much data is inside.

3. Understanding "Occupancy"

Occupancy refers to how "full" the FIFO is at any given moment—essentially the **Data Count**.

Because this is an asynchronous FIFO, both sides need to know the occupancy for different reasons:

- **Write Occupancy (*wdcnt_o*)**: Tells the sender how many words are currently stored.
- **Read Occupancy (*rdcnt_o*)**: Tells the receiver how many words are available to be

read.

The Occupancy Logic (WDCOUNT_CAL** and **RDCOUNT_CAL**):** Calculating occupancy isn't always a simple subtraction because these are circular buffers. The code checks the Most Significant Bit (MSB) to see if the pointers have "wrapped around".

- **Normal Case:** If both pointers are in the same "lap," the occupancy is simply **Write_Pointer - Read_Pointer**.
- **Wrap-Around Case:** If the write pointer has looped back to the start but the read pointer hasn't, it calculates the distance across the "boundary" using the **DEPTH** of the FIFO.

Summary of Internal Signals

Signal	Purpose
wbin_r / rbin_r	Local binary pointers for RAM addressing.
wptr_r / rptr_r	Gray code versions of pointers for safe travel.
wrptr_sync	The Read Pointer, safely arrived in the Write Domain.
rwptr_sync	The Write Pointer, safely arrived in the Read Domain.
wfull_o	The "Stop" sign for the writer.
rempty_o	The "Wait" sign for the reader.

This asynchronous FIFO design is built to manage data flow between two different clock domains while preventing data loss or corruption. Below is the detailed breakdown of every block in the code, explaining the logic and the hardware it represents.

1. Generic and Port Block

The **entity** block defines the physical interface and configurable parameters of the hardware.

- Generics : * **ADDR_BITS**: Set to 9, meaning the FIFO has an address space of 2^9 , or 512 locations.
 - **DATA_BITS**: Set to 32, defining the width of each data word stored in the RAM.
- Ports:
 - **Write Domain (**wclk_i**, **wrst_n_i**)**: All write-side signals are synchronized to this

- clock.
 - **Read Domain (rclk_i, rrst_n_i)**: All read-side signals are synchronized to this clock.
 - **Data/Control**: wreq_i and rreq_i are the "push" and "pop" commands. wfull_o and rempty_o are the safety flags.
 - **Occupancy Ports**: wdcnt_o and rdcnt_o provide the real-time count of data currently inside the FIFO from both perspectives.
-

2. Constant and Signal Declaration

These define the internal "wires" and "registers" used within the architecture.

- **DEPTH**: A constant set to $2^{\text{ADDR_BITS}}$ (512) used for occupancy math.
 - **Pointers (wbin_r, rbin_r, etc.)**: These are 10-bit signals (ADDR_BITS downto 0). The extra 10th bit is crucial; it is the **wrap-around bit** used to distinguish if the FIFO is empty (pointers match) or full (pointers match except for the MSB).
 - **Gray Pointers (wptr_r, rptr_r)**: These carry the Gray-coded version of the binary pointers across clock domains.
 - **Synchronizer Chain (meta and sync)**: These signals (like wrptr_meta and wrptr_sync) represent a **two-stage flip-flop synchronizer**. This is the "bridge" that prevents metastability when a signal moves from the read clock to the write clock.
 - **ram_type**: An array that represents the physical memory storage.
-

3. Function Block (XOR Reduction)

The function XOR_Reduction is a mathematical utility used to convert Gray code back into Binary.

- **Logic**: It takes an input vector and XORs every bit from the specified index to the Most Significant Bit (MSB).
 - **Why it's here**: Gray code is used for safety during clock-domain crossing. However, you cannot perform subtraction on Gray code to find the data count. Therefore, the code uses this function to turn the "foreign" Gray pointer back into a Binary number (wrbin and rwbins) so it can perform math on it.
-

4. Sequential Block (Clocked Logic)

These blocks represent physical registers (Flip-Flops) that update only on a clock edge.

- **REG_WRT_WCLK_I :**
 - Resets the write pointers and **wfull_o** flag to '0' .
 - Updates the local write pointers (**wptr_r**, **wbin_r**) and the **wfull_o** flag .
 - Implements the two-stage synchronization of the read pointer into the write domain.
- **REG_WRT_RCLK_I :**
 - Resets the read pointers and sets **empty_o** to '1' (because the FIFO starts empty) .
 - Synchronizes the write pointer into the read domain.
- **RAM_WR & RAM_RD :** These are the memory access ports. Writing occurs if **wreq_i** is active. Reading occurs if **rreq_i** is active, updating **rdata_o**.

5. Combinational Block (Continuous Logic)

This block calculates values instantly based on current signal states.

- **Pointer Advancement:**
 - **wbin_nxt** is simply the current pointer plus the request bit.
 - **wptr_nxt** converts that binary result into Gray code using the XOR-shift method:
$$\$(Bin \setminus gg \ 1) \oplus Bin\$$$
- **Flag Generation:**
 - **Empty:** **empty_nxt** is high if the next read pointer matches the synchronized write pointer.
 - **Full:** **wfull_nxt** uses bit-inversion logic on the MSBs to see if the write pointer has wrapped around the buffer and caught the read pointer.
- **Occupancy Calculation (WDCOUNT_CAL & RDCOUNT_CAL) :**
 - **"Occupancy"** is the measure of how much data is currently in the FIFO (Data Count).
 - The logic compares the local pointer with the synchronized pointer from the other side.
 - If the pointers are in the same "lap" (same MSB), it performs a direct subtraction.
 - If they are in different "laps" (wrap-around), it subtracts the difference from the **DEPTH** to find the true count .

1. Generic and Port Block

This block defines the "template" and the "wires" connecting the FIFO to the outside world.

- **Generics :**
 - **ADDR_BITS (9):** Determines the depth. $2^{\{ADDR_BITS\}}$ gives us 512

- locations .
- **DATA_BITS (32)**: The width of the data bus.
- Ports :
 - **Write Domain (wclk_i, wrst_n_i, wreq_i, wdata_i)**: Signals that handle incoming data.
 - **Read Domain (rclk_i, rst_n_i, req_i, rdata_o)**: Signals that handle outgoing data.
 - **Status/Count**: wfull_o and rempty_o prevent overflow/underflow, while wdcnt_o and rdcnt_o report occupancy .

2. Constant and Signal Declaration

These are the internal "storage" and "wires" that make the logic work.

- **DEPTH** : A constant ($2^9 = 512$) used for occupancy calculations.
- **ram_type** & **ram** : This defines the memory structure. It is an array of 512 rows, each 32 bits wide.
- **Pointers** : Both binary (**bin**) and Gray code (**ptr**) versions exist. They are 10 bits wide (**ADDR_BITS downto 0**) .
- **Synchronizers** : The **meta** and **sync** signals form a **Two-Stage Flip-Flop Synchronizer** to prevent metastability when crossing clock domains.

3. The RAM Block (Internal Memory)

The RAM is implemented as **Inferred Dual-Port Distributed RAM**. It is split into two distinct processes to handle the two different clocks.

RAM Write (**RAM_WR**)

- **Logic**: This process is sensitive only to **wclk_i**.
- **Function**: On the rising edge of the write clock, it checks the **wreq_i** (Write Request) signal .
- **Addressing**: If **wreq_i** is '1', it stores **wdata_i** into the **ram** array.
- **The Address Trick**: It uses **wbin_r(ADDR_BITS-1 downto 0)**. It ignores the 10th "wrap-around" bit and only uses the lower 9 bits to select one of the 512 physical memory slots.

RAM Read (**RAM_RD**)

- **Logic**: This process is sensitive to **rclk_i**.
- **Function**: On the rising edge of the read clock, it checks **rreq_i** (Read Request) .
- **Addressing**: It fetches the data from the **ram** at the address specified by the lower 9 bits

of the read pointer (`rbin_r`) and assigns it to `rdata_o`.

4. Function Block: XOR Reduction

The `XOR_Reduction` function is the mathematical engine that converts Gray code back to Binary.

- **The Logic:** To convert Gray to Binary, you XOR a bit with all bits to its left.
- **Example:** If you have a Gray code G , the Binary bit $B[i]$ is calculated as:
$$B[i] = G[i] \oplus G[i+1] \oplus \dots \oplus G[\text{MSB}]$$
- **The Code:** It uses a loop to XOR the data vector from index i up to the `ADDR_BITS`. This is essential because while Gray is safe for traveling across clocks, you can't do subtraction with it.

5. Sequential Block (Clocked Logic)

These blocks update the state of the FIFO on every clock tick.

- `REG_WRT_WCLK_I` : Manages the write side. It updates write pointers and synchronizes the incoming read pointer .
- `REG_WRT_RCLK_I` : Manages the read side. It updates read pointers and synchronizes the incoming write pointer .
- **Synchronization:** Both use the `meta` register to catch the pointer and the `sync` register to stabilize it .

6. Combinational Block (Continuous Math)

This is where the "intelligence" of the FIFO lives.

- **Pointer Advance:** `wbin_nxt` and `rbin_nxt` are calculated by adding the request bit (0 or 1) to the current pointer.
- **Binary to Gray:** `wptr_nxt` is calculated as $(\text{Binary} \gg 1) \text{ XOR Binary}$.
- **Occupancy (Data Count):**
 - **Definition:** "Occupancy" is how many words are currently stored.
 - **Logic (`WDCOUNT_CAL` / `RDCOUNT_CAL`):** It compares the local pointer with the synchronized pointer from the other side .
 - If the MSBs (the 10th bits) are the same, it is a simple subtraction .
 - If the MSBs are different (meaning one pointer wrapped around), it calculates the distance across the "lap" using the `DEPTH` constant .